

Module 3 Introduction to OOPS Programming

Introduction to C++

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

➔ Key Differences Between Procedural Programming and Object-Oriented Programming (OOP)

Procedural Programming (POP)	Object-Oriented Programming (OOP)
Program is divided into functions.	Program is divided into classes and objects.
Data and functions are separate.	Data and functions are combined in a class.
Follows a top-down approach.	Follows a bottom-up approach.
Less secure, data can be accessed easily.	More secure because of encapsulation and data hiding.
Example languages: C, Pascal	Example languages: C++, Java, Python

2. List and explain the main advantages of OOP over POP.

➔ Advantages of Object-Oriented Programming (OOP) over Procedural Programming (POP)

1. **Encapsulation (Data Hiding)**

- In OOP, data and functions are combined in a **class**, which helps protect data from unauthorized access.

2. **Reusability**

- OOP allows **code reuse** through concepts like **inheritance**, so the same code can be used in multiple programs.

3. Modularity

- Programs are divided into **small classes and objects**, making them easier to manage and understand.

4. Easy Maintenance

- OOP programs are easier to **modify and update** because changes in one class do not affect the whole program.

5. Better Security

- Data can be **hidden using access specifiers** (private, protected, public), improving program security.

6. Real-World Modeling

- OOP represents real-world objects (like **car, student, bank account**) making programs more logical and easier to design.

3.) Explain the steps involved in setting up a C++ development environment.

➔ Steps to Set Up a C++ Development Environment

1. Install an IDE

- Download and install a C++ IDE such as **Dev C++** or **CodeBlocks** on your computer.

2. Install a Compiler

- Ensure a C++ compiler (like **GCC** or **MinGW**) is installed. Most IDEs include a compiler.

3. Create a New Project/File

- Open the IDE and create a **new C++ project** or **source file (.cpp)**.

4. Write the Program

- Type the C++ code in the editor.

5. Compile the Program

- Click **Compile** or **Build** to convert the code into an executable file.

6. Run the Program

- Click **Run** to execute the program and see the output.

4.) What are the main input/output operations in C++?
Provide examples.

➔ Main Input/Output Operations in C++

In C++, **Input/Output (I/O)** operations are used to **take data from the user and display results on the screen**. These operations are performed using the **iostream** library.

1. Output Operation (cout)

- **cout** is used to display output on the screen.

Example:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World";
    return 0;
}
```

Output:

Hello World

2. Input Operation (cin)

- **cin** is used to take input from the user.

Example:

```
#include <iostream>
using namespace std;

int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;

    cout << "You entered: " << num;
    return 0;
}
```

Output :

```
Enter a number:12
You entered:12
```

5.) What are the different data types available in C++? Explain with examples.

➔ 1. Basic (Primitive) Data Types

These are the most commonly used data types.

Data Type	Description	Example
Int	Stores integer numbers	int a = 10;
float	Stores decimal numbers	float b = 5.5;
double	Stores large decimal numbers with more precision	double c = 10.12345;
Char	Stores a single character	char grade = 'A';
Bool	Stores true or false values	bool isPassed = true;
Void	Represents no value	void display();

Example Program

```
#include<iostream>
using namespace std;

int main() {
    int a = 10;
    float b = 5.5;
    char c = 'A';
    bool d = true;

    cout << a << endl;
    cout << b << endl;
    cout << c << endl;
    cout << d << endl;

    return 0;
}
```

2. Derived Data Types

These data types are derived from basic data types.

Examples:

- **Array**
- **Pointer**
- **Reference**
- **Function**

Example

```
int arr[5] = {1,2,3,4,5};    // Array
int *ptr = &arr[0];          // Pointer
```

3. User-Defined Data Types

These are created by the programmer.

Examples:

- struct
- union

- enum
- class

Example

```
struct Student {  
    int roll;  
    char name[20];  
};
```

6.) Explain the difference between implicit and explicit type conversion in C++.

➔ In C++, **type conversion** means converting one data type into another. It can be done in two ways: **Implicit Type Conversion** and **Explicit Type Conversion**.

1. Implicit Type Conversion (Automatic Conversion)

- It is done **automatically by the compiler**.
- Occurs when **different data types are used in an expression**.
- Usually converts **smaller data types into larger data types** to avoid data loss.

Explanation:

Here, a is an **int**, but during calculation it is automatically converted to **float**.

2. Explicit Type Conversion (Type Casting)

- It is done **manually by the programmer**.
- Used when the programmer **forces one data type to convert into another**.

Explanation:

Here (float) a converts integer a into **float** so that the division gives a **decimal result**.

Feature	Implicit Conversion	Explicit Conversion
Who performs it	Compiler	Programmer
Also called	Automatic conversion	Type casting
Syntax needed	No	Yes ((type))
Risk of data loss	Very low	Possible

7.) What are the different types of operators in C++?
Provide examples of each.

→1. Arithmetic Operators

Used to perform mathematical operations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b

2. Relational Operators

Used to compare two values. The result is **true or false**.

Operator	Meaning
----------	---------

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

3. Logical Operators

Used to combine conditions.

Operator	Meaning
&&	Logical AND
^	
!	Logical NOT

4. Assignment Operators

Used to assign values to variables.

Operator	Example
=	a = 5
+=	a += 3

--=	a -= 2
*=	a *= 4
/=	a /= 2

5. Increment and Decrement Operators

Operator	Meaning
++	Increase value by 1
--	Decrease value by 1

6. Conditional (Ternary) Operator

Used to make decisions in a single line.

Syntax

```
condition ? expression1 : expression2;
```

8.) Explain the purpose and use of constants and literals in C++.

➔ Constants

A **constant** is a variable whose **value cannot be changed during the execution of a program**. Once a constant is defined, it remains the same throughout the program. Constants are generally declared using the `const` keyword.

➤ Purpose of Constants

The main purpose of constants is to store values that **should remain fixed** during the execution of the program. For example, values like the value of **PI**,

the number of days in a week, or a fixed tax rate should not change while the program runs.

➤ Use of Constants

Constants are used in programs to:

- Prevent **accidental modification** of important values.
- Improve **program readability**, because meaningful names can be given to constant values.
- Make programs **easier to maintain**, since changing the constant in one place updates it everywhere it is used.
- Provide **better program reliability and safety**.

For example, if the value of PI is used many times in a program, defining it as a constant ensures that the value stays the same everywhere in the program.

➤ Literals

A **literal** is a **fixed value written directly in the program code**. Unlike constants, literals do not have names; they are simply values used in expressions or assigned to variables.

➤ Purpose of Literals

The purpose of literals is to **represent constant values directly** in the program. They are used whenever a specific value is needed in a calculation or assignment.

➤ Types of Literals in C++

- **Integer Literals**
These represent whole numbers without decimal points, such as 10, 25, or 100.
- **Floating-Point Literals**
These represent numbers with decimal points, such as 3.14 or 5.75.
- **Character Literals**
These represent a single character enclosed in single quotes, such as 'A' or 'b'.
- **String Literals**
These represent a sequence of characters enclosed in double quotes, such as "Hello" or "Programming".

- **Boolean Literals**

These represent logical values such as **true** and **false**.

- Use of Literals

Literals are used to:

- Assign values directly to variables.
 - Perform calculations in expressions.
 - Represent fixed data values needed in the program.
-

9.) What are conditional statements in C++? Explain the if-else and switch statements.

➔ Conditional Statements in C++

Conditional statements are used to **control the flow of a program based on conditions**.

They help the program make decisions and execute different parts of code depending on whether a condition is **true or false**.

◆ if-else Statement (Theory)

The **if-else statement** is used when a program needs to choose between two or more alternatives.

- The **if part** executes when the condition is **true**
- The **else part** executes when the condition is **false**.
- It works with **any type of condition** (>, <, ==, !=, etc.)
- It is useful for **complex decision-making**
- It can be extended using **else-if ladder** for multiple conditions

◆ else-if Ladder:

- Used when there are **multiple conditions to check**
- Conditions are checked **one by one**
- The first true condition is executed, and the rest are skipped

◆ switch Statement (Theory)

The **switch statement** is used when a variable can have **multiple fixed values**, and different actions are needed for each value.

- It works with **integer or character values**
- It compares the value of an expression with different **cases**
- Each case represents a possible value
- The **break statement** is used to stop execution after a match
- The **default case** runs when no case matches

10.) What is the difference between for, while, and do-while loops in C++?

1. for Loop

The **for loop** is used when the number of iterations is **known in advance**.

◆ Key Points:

- Initialization, condition, and increment/decrement are written **in one line**
- Best suited for **counting loops**
- Easy to read and manage

2. while Loop

The **while loop** is used when the number of iterations is **not known beforehand**.

◆ Key Points:

- Condition is checked **before execution**
 - Loop runs **only if the condition is true**
 - Can execute **zero or more times**
-

◆ 3. do-while Loop

The **do-while loop** is similar to the while loop but with one important difference.

◆ Key Points:

- Condition is checked **after execution**
- Loop runs **at least once**, even if the condition is false
- Useful when at least one execution is required

Main Differences

Feature	for Loop	while Loop	do-while Loop
Condition Check	Before execution	Before execution	After execution
Execution Count	Known (fixed)	Unknown	Unknown
Minimum Execution	0 times	0 times	At least 1 time
Structure	All in one line	Separate initialization	Similar to while
Use Case	Counting loops	Condition-based loops	Menu-driven programs

11.)

11.)How are break and continue statements used in loops?
Provide examples.

➔1. break Statement

The **break statement** is used to **immediately exit the loop** when a condition is met.

◆ Key Points:

- Terminates the loop completely
- Control moves to the statement **after the loop**
- Used in loops and also in **switch statements**

◆ Example:

```
#include<iostream>
using namespace std;

int main() {
    for(int i = 1; i <= 10; i++) {
        if(i == 5) {
            break;    // loop stops when i = 5
        }
        cout << i << " ";
    }
    return 0;
}
```

Output:

1 2 3 4

Loop stops as soon as i becomes 5.

2. continue Statement

The **continue statement** is used to **skip the current iteration** and move to the next iteration of the loop.

◆ Key Points:

- Does NOT stop the loop
- Skips only the current iteration
- Loop continues with the next value

◆ Example:

```
#include<iostream>
using namespace std;

int main() {
    for(int i = 1; i <= 10; i++) {
```

```
        if(i == 5) {
            continue;    // skip when i = 5
        }
        cout << i << " ";
    }
    return 0;
}
```

Output:

1 2 3 4 6 7 8 9 10

Number 5 is skipped, but the loop continues.

12.) Explain nested control structures with an example.

➔ Nested Control Structures in C++

Nested control structures mean placing one control statement **inside another control statement**.

This allows a program to make **more complex decisions** or perform **repeated operations within another loop or condition**.

◆ Types of Nested Structures

1. **Nested if** → if statement inside another if
 2. **Nested loops** → loop inside another loop
 3. **Combination** → loop inside if or if inside loop
-

◆ Key Points:

- Used for **complex logic and multi-level conditions**
 - Inner structure executes **only when outer condition is satisfied**
 - Common in problems like patterns, matrices, and decision-making
-

◆ Example (Nested if)

```
#include<iostream>
using namespace std;

int main() {
    int num = 10;

    if(num > 0) {
        if(num % 2 == 0) {
            cout << "Positive Even Number";
        }
    }

    return 0;
}
```

First condition checks if the number is positive
Second condition (inside) checks if it is even

◆ Example (Nested loop)

```
#include<iostream>
using namespace std;

int main() {
    for(int i = 1; i <= 3; i++) {
        for(int j = 1; j <= 3; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

◆ Output:

```
* * *
* * *
* * *
```

Outer loop controls rows
Inner loop controls columns

13.) What is a function in C++? Explain the concept of function declaration, definition, and calling.

➔ A **function** in C++ is a **block of code** that performs a specific task. It helps in **reusing code**, improving readability, and making programs easier to manage.

Instead of writing the same code again and again, we can use functions.

◆ Concept of Function Declaration, Definition, and Calling

◆ 1. Function Declaration (Prototype)

A **function declaration** tells the compiler:

- Function name
- Return type
- Parameters (arguments)

☞ It is written **before the main function**.

◆ Key Points:

- No function body is written here
- Ends with a semicolon (;)
- Helps the compiler know about the function in advance

◆ 2. Function Definition

A **function definition** contains the **actual code (body)** of the function.

◆ Key Points:

- Specifies what the function does
- Includes statements to perform the task
- Written either before or after `main()`

◆ 3. Function Calling

A **function call** is used to **execute the function**.

◆ Key Points:

- Written inside `main()` or another function
 - Passes values (arguments) if required
 - Transfers control to the function
-

14.) What is the scope of variables in C++? Differentiate between local and global scope.

➔ Scope of Variables in C++

The **scope of a variable** refers to the **area or region in a program where the variable can be accessed or used**.

In simple words, it defines **where a variable is visible and usable** in the program.

◆ Types of Scope in C++

1. **Local Scope**
 2. **Global Scope**
-

◆ 1. Local Scope

A variable declared **inside a function or block** has **local scope**.

◆ Key Points:

- Accessible **only within the function or block** where it is declared
 - Cannot be used outside that function
 - Created when the function starts and destroyed when it ends
-

◆ 2. Global Scope

A variable declared **outside all functions** has **global scope**.

◆ Key Points:

- Accessible **throughout the program** (in all functions)
- Exists for the **entire lifetime of the program**
- Can be used by multiple functions

▼ Difference Between Local and Global Scope

Feature	Local Scope	Global Scope
Declaration Place	Inside function/block	Outside all functions
Accessibility	Limited to that function/block	Accessible everywhere
Lifetime	Temporary (during function execution)	Entire program execution
Usage	For specific tasks	For shared data

15.) Explain recursion in C++ with an example.

➔ Recursion in C++

Recursion is a technique in C++ where a **function calls itself** to solve a problem.

☞ It is mainly used for problems that can be broken down into **smaller sub-problems of the same type**.

◆ Key Concepts of Recursion

1. Base Case

- The condition where recursion **stops**
- Prevents infinite looping

2. Recursive Case

- The part where the function **calls itself** with a smaller input

◆ How Recursion Works

- The function keeps calling itself
- Each call works on a smaller problem
- When the base case is reached, the function stops and returns results back step by step

◆ Example: Factorial of a Number

☞ Factorial formula:

$$n! = n \times (n-1)!$$

◆ C++ Example:

```
#include<iostream>
using namespace std;

int factorial(int n) {
    if(n == 0) {                // Base case
        return 1;
    }
    else {                      // Recursive case
        return n * factorial(n - 1);
    }
}

int main() {
    int num = 5;
    cout << "Factorial = " << factorial(num);
    return 0;
}
```

16.) What are function prototypes in C++? Why are they used?

➔ Function Prototypes in C++

A **function prototype** is a **declaration of a function** that tells the compiler about:

- Function name
- Return type
- Number and type of parameters

☞ It is written **before the main function** and does not contain the function body.

◆ Example of Function Prototype

```
int add(int, int);
```

◆ Why are Function Prototypes Used?

Function prototypes are important for the following reasons:

◆ 1. Inform the Compiler

- They tell the compiler that a function **exists** even if it is defined later
- This avoids errors when the function is called before its definition

◆ 2. Type Checking

- Ensures that the **correct number and type of arguments** are passed
- Helps in catching errors at compile time

◆ 3. Improves Code Organization

- Allows programmers to **separate function declaration and definition**
- Makes programs more **readable and modular**

❖ 4. Supports Function Reusability

- Functions can be defined **anywhere in the program or in other files**
- Prototype ensures proper linking and usage

▼ Key Points

- Ends with a semicolon (;)
 - Does not contain function body
 - Also called **function declaration**
 - Must match with function definition
-

17.) What are arrays in C++? Explain the difference between single-dimensional and multi- dimensional arrays.

➔ What are Arrays in C++?

An **array** in C++ is a **collection of elements of the same data type** stored in **contiguous memory locations**.

☞ It allows you to store multiple values in a single variable instead of creating separate variables.

❖ Key Points:

- All elements must be of the **same data type**
- Elements are accessed using an **index (position)**
- Index starts from **0**
- Size of the array is **fixed**

❖ Types of Arrays

1. **Single-Dimensional Array**
 2. **Multi-Dimensional Array**
-

◆ 1. Single-Dimensional Array

A **single-dimensional array** is a **linear list of elements** stored in a single row.

◆ Key Points:

- Uses **one index** to access elements
- Represents a **simple list or sequence**
- Easy to use and understand

☞ Example: List of marks, numbers, etc.

◆ 2. Multi-Dimensional Array

A **multi-dimensional array** is an array of arrays.
It is used to store data in **tabular form (rows and columns)**.

◆ Key Points:

- Uses **two or more indices**
- Commonly used as **2D arrays (matrix form)**
- Represents data like tables, grids, matrices

☞ Example: Marks of students in different subjects

▼ Difference Between Single-Dimensional and Multi-Dimensional Arrays

Feature	Single-Dimensional Array	Multi-Dimensional Array
Structure	Linear (one row)	Table (rows and columns)
Number of Indices	One	Two or more
Data Representation	List	Matrix/Table
Complexity	Simple	More complex

Example Use	Storing marks	Storing matrix data
-------------	---------------	---------------------

18.) Explain string handling in C++ with examples.

➔ String Handling in C++

In C++, a **string** is a sequence of characters used to store text. C++ provides **two ways** to handle strings:

1. **C-style strings** (character arrays)
2. **C++ strings** (using `string` class from `<string>` library)

◆ 1. C-Style Strings

- A **C-style string** is an **array of characters** ending with a **null character** `\0`.
- Example: "Hello" is stored as `{ 'H', 'e', 'l', 'l', 'o', '\0' }`.

◆ Example:

```
#include<iostream>
using namespace std;

int main() {
    char name[20] = "Kenil";

    cout << "Name: " << name << endl;

    return 0;
}
```

◆ Output:

Name: Kenil

◆ Common Operations:

- `strlen(name)` → length of string
- `strcpy(dest, src)` → copy string
- `strcat(str1, str2)` → concatenate strings
- `strcmp(str1, str2)` → compare strings

❖ 2. C++ Strings (String Class)

- C++ provides a **string class** in the `<string>` library
- Easier to use and more **flexible than C-style strings**

❖ Example:

```
#include<iostream>
#include<string>
using namespace std;

int main() {
    string name = "Kenil";

    cout << "Name: " << name << endl;

    // String concatenation
    string surname = " Marwawala";
    string fullname = name + surname;
    cout << "Full Name: " << fullname << endl;

    // String length
    cout << "Length: " << fullname.length() << endl;

    return 0;
}
```

❖ Output:

```
Name: Kenil
Full Name: Kenil Marwawala
Length: 14
```

19.) How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

➔ Array Initialization in C++

Array initialization means assigning **values to an array at the time of declaration**.

Arrays can be **1D (single-dimensional)** or **2D (multi-dimensional)**.

◆ 1. Initializing 1D Array

A **1D array** is a simple linear array.

◆ Syntax:

```
data_type array_name[size] = {value1, value2, ..., valueN};
```

◆ Example:

```
#include<iostream>
using namespace std;

int main() {
    int marks[5] = {80, 90, 75, 85, 95};

    cout << "Marks: ";
    for(int i = 0; i < 5; i++) {
        cout << marks[i] << " ";
    }

    return 0;
}
```

◆ Output:

Marks: 80 90 75 85 95

◆ Notes:

- If you provide **fewer values than size**, remaining elements are **initialized to 0**.
- Example: `int arr[5] = {1, 2};` → `arr = {1, 2, 0, 0, 0}`

◆ 2. Initializing 2D Array

A **2D array** is like a table with **rows and columns**.

◆ Syntax:

```
data_type array_name[rows][columns] = {
    {value11, value12, ..., value1N},
    {value21, value22, ..., value2N},
    ...
};
```

◆ Example:

```
#include<iostream>
using namespace std;

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    cout << "Matrix:" << endl;
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

◆ Output:

```
Matrix:
1 2 3
4 5 6
```

20.) Explain string operations and functions in C++.

➔ String Operations in C++

In C++, a **string** is a sequence of characters.

Strings can be handled using **C-style character arrays** or **C++ string class**.

Common string operations include:

◆ 1. String Input and Output

- **Input** using `cin` or `getline()`
- **Output** using `cout`

Example:

```
#include<iostream>
#include<string>
using namespace std;

int main() {
    string name;

    cout << "Enter your name: ";
    getline(cin, name); // allows spaces

    cout << "Hello, " << name << endl;

    return 0;
}
```

❖ 2. String Concatenation

- Join two strings together
- Use **+** operator or **append()** function

Example:

```
string first = "Kenil";
string last = " Marwawala";

string fullname = first + last;           // Using +
fullname.append(" C++");                 // Using
append()

cout << fullname;                         // Output:
Kenil Marwawala C++
```

❖ 3. String Length

- Find the number of characters using **length()** or **size()**

Example:

```
string str = "Hello World";
cout << str.length();    // Output: 11
cout << str.size();      // Output: 11
```

◆ 4. Accessing Characters

- Access individual characters using **indexing []**
- Index starts from **0**

Example:

```
string str = "Hello";  
cout << str[0];    // Output: H  
str[0] = 'h';      // Change character  
cout << str;       // Output: hello
```

◆ 5. String Comparison

- Compare strings using **==, != operators**
- Or using **compare ()** function

Example:

```
string str1 = "apple";  
string str2 = "orange";  
  
if(str1 == str2)  
    cout << "Equal";  
else  
    cout << "Not Equal";    // Output: Not Equal  
  
// Using compare()  
cout << str1.compare(str2); // Negative value,  
since "apple" < "orange"
```

◆ 6. Substring Extraction

- Extract part of a string using **substr(start, length)**

Example:

```
string str = "Hello World";  
string part = str.substr(0,5); // Extract "Hello"  
cout << part;
```

21.) Explain the key concepts of Object-Oriented Programming (OOP).

➔ Object-Oriented Programming (OOP) in C++

Object-Oriented Programming (OOP) is a programming paradigm that **organizes software around objects** rather than functions or logic.

- **Object:** Represents a real-world entity with **attributes (data)** and **behaviors (methods)**.
- **Class:** A blueprint or template to create objects.

OOP helps in **code reusability, modularity, and easy maintenance**.

◆ Key Concepts of OOP

1. Class

- A **class** is a blueprint for creating objects.
- It defines **data members (attributes)** and **member functions (behaviors)**.

Example:

```
class Car {  
    public:  
        string brand;  
        int speed;  
        void drive() {  
            cout << "Car is driving";  
        }  
};
```

2. Object

- An **object** is an **instance of a class**.
- It has its **own values for the attributes** and can use class methods.

Example:

```
Car myCar;           // object of class Car
myCar.brand = "Honda";
myCar.speed = 100;
myCar.drive();
```

3. Encapsulation

- **Encapsulation** means **binding data and methods together** inside a class.
- Helps in **data hiding** using access specifiers:
 - `private` → hidden from outside
 - `public` → accessible outside
 - `protected` → accessible in derived classes

Benefit: Prevents unauthorized access to data.

4. Inheritance

- **Inheritance** allows a class (derived) to **inherit properties and behaviors** from another class (base).
- Promotes **code reuse**.

Example:

```
class Vehicle { public: int wheels; };
class Car : public Vehicle { public: string brand; };
```

- Car inherits wheels from Vehicle.
-

5. Polymorphism

- **Polymorphism** means “many forms.”
- It allows a **single function or operator** to behave differently in different contexts.

Types:

1. **Compile-time (Function Overloading / Operator Overloading)**
2. **Run-time (Virtual Functions / Method Overriding)**

Example (Function Overloading):

```
int add(int a, int b);  
float add(float a, float b);
```

6. Abstraction

- **Abstraction** means **hiding unnecessary details** and showing only **essential features**.
- Achieved using:
 - **Abstract classes**
 - **Pure virtual functions**

Example:

```
class Shape {  
    public:  
        virtual void draw() = 0;    // pure virtual  
function  
};
```

22.) What are classes and objects in C++? Provide an example.

➔ Classes and Objects in C++

1. Class

- A **class** is a **blueprint or template** that defines the **data members (attributes)** and **member functions (behaviors)** of objects.
- It **does not occupy memory** by itself.
- Think of it as a **plan for creating objects**.

Syntax:

```
class ClassName {  
    access_specifier:  
        data members;        // variables  
        member functions;    // functions  
};
```

2. Object

- An **object** is an **instance of a class**.
- It **occupies memory** and contains actual values for the class attributes.
- Objects can **use the member functions** of their class.

Example Analogy:

- Class → Blueprint of a house
- Object → A real house built using that blueprint

◆ Example in C++

```
#include<iostream>
using namespace std;

// Class Declaration
class Car {
    public:
        string brand;
        int speed;

        void drive() {                // Member Function
            cout << brand << " is driving at " <<
speed << " km/h" << endl;
        }
};

int main() {
    // Creating Object
    Car myCar;

    // Accessing Members
    myCar.brand = "Honda";
    myCar.speed = 120;

    myCar.drive(); // Calling Member Function

    return 0;
}
```

◆ Output:

Honda is driving at 120 km/h

23.) What is inheritance in C++? Explain with an example.

➔ Inheritance in C++

Inheritance is a feature of **Object-Oriented Programming (OOP)** where a **new class (derived class)** acquires the **properties and behaviors of an existing class (base class)**.

- Helps in **code reuse**
- Promotes **hierarchical classification**
- Avoids rewriting the same code in multiple classes

◆ Key Points:

1. **Base Class (Parent Class)** → The class whose properties and methods are inherited.
2. **Derived Class (Child Class)** → The class that inherits properties and methods.
3. **Access Specifiers** affect inheritance:
 - `public` inheritance → public members of base become public in derived
 - `protected` inheritance → public members become protected
 - `private` inheritance → public members become private

◆ Syntax:

```
class Base {  
    // members  
};  
  
class Derived : access_specifier Base {  
    // additional members  
};
```

◆ Example of Inheritance

```
#include<iostream>  
using namespace std;  
  
// Base Class  
class Vehicle {
```

```

    public:
        int wheels;
        void displayWheels() {
            cout << "Wheels: " << wheels << endl;
        }
};

// Derived Class
class Car : public Vehicle {
    public:
        string brand;
        void displayBrand() {
            cout << "Brand: " << brand << endl;
        }
};

int main() {
    Car myCar;

    // Accessing Base Class Members
    myCar.wheels = 4;
    myCar.displayWheels();

    // Accessing Derived Class Members
    myCar.brand = "Honda";
    myCar.displayBrand();

    return 0;
}

```

◆ Output:

```

Wheels: 4
Brand: Honda

```

24.) What is encapsulation in C++? How is it achieved in classes?

➔ Encapsulation in C++

Encapsulation is one of the key concepts of **Object-Oriented Programming (OOP)**.

Definition:

Encapsulation is the **wrapping of data (variables) and methods (functions) together into a single unit**, usually a **class**, and **restricting direct access to some of the class members**.

- Also called **data hiding**
- Helps **protect data from unauthorized access**
- Ensures **controlled access** through **member functions**

◆ How Encapsulation is Achieved in C++

Encapsulation is implemented in C++ using **classes and access specifiers**:

1. Private Members

- Declared using `private`
- **Cannot be accessed directly** outside the class

2. Public Members

- Declared using `public`
- **Accessible outside the class**
- Used to **access or modify private data safely** through functions (getters and setters)

◆ Example of Encapsulation

```
#include<iostream>

using namespace std;

// Class with Encapsulation
class Student {
    private:                // Private data (hidden)
        int marks;
```

```

        public:                // Public functions
(controlled access)

        void setMarks(int m) {    // Setter
            if(m >= 0 && m <= 100)
                marks = m;
            else
                cout << "Invalid marks!" << endl;
        }

        int getMarks() {        // Getter
            return marks;
        }
};

int main() {
    Student s1;

    s1.setMarks(85);
    cout << "Student marks: " << s1.getMarks() << endl;

    return 0;
}

```

Output:

Student marks: 85
